



# Generative Text

# This Week's Agenda

## AI News

### Generative Language Models

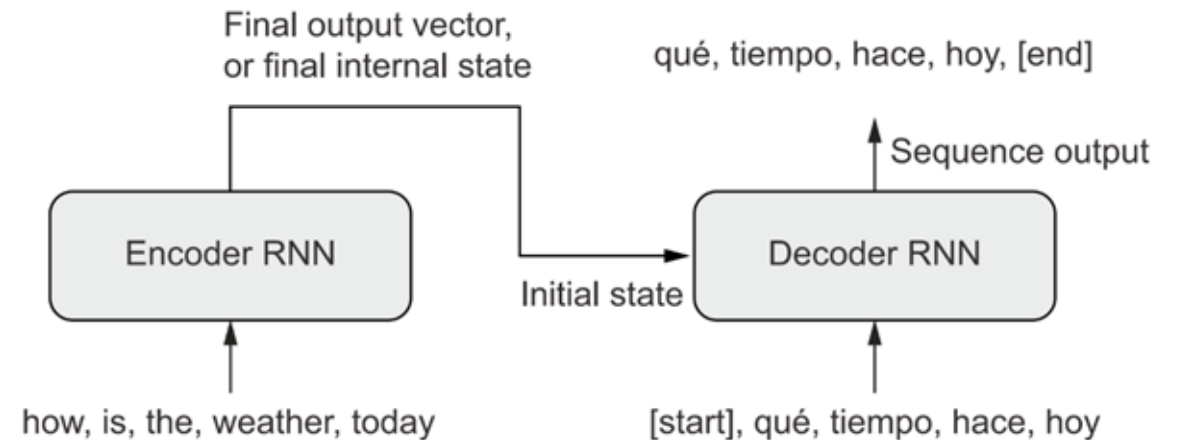
- Machine translation
- Large language models
- Auto-regressive (large) language models
- Pre-trained LLMs
- LLM + RAG



# Machine Translation: Encoder-Decoder

## Similar Idea with Some Additions

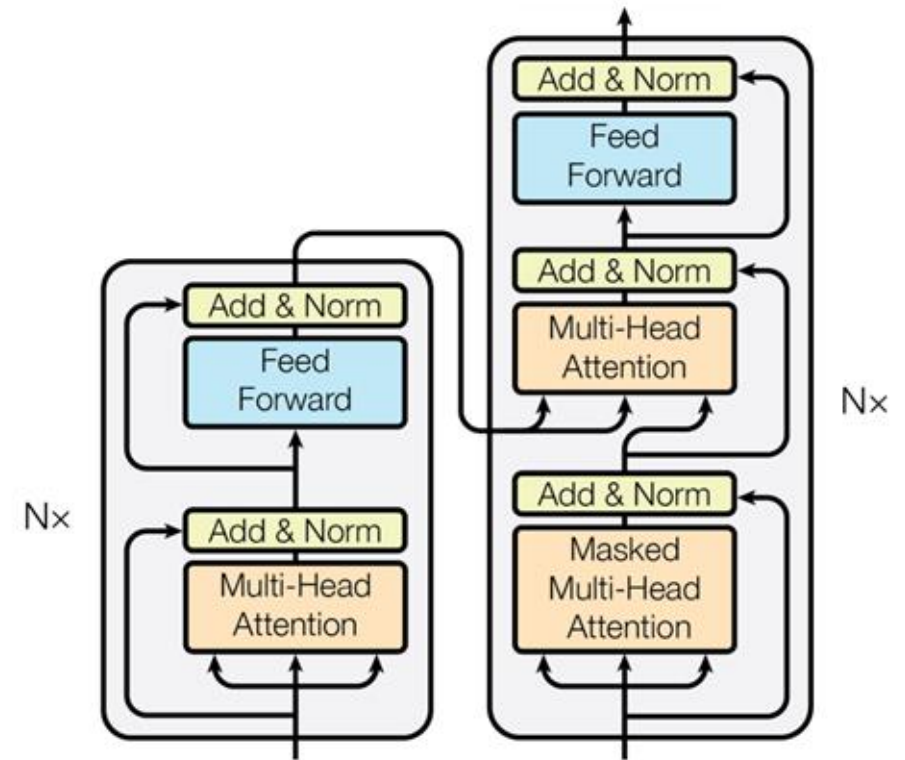
- Include an 'encoder' that first processes the entire input sequence to obtain an embedded representation of the input text.
- We are not just generating 'plausible' text now; we are generating plausible text that aligns with (i.e., conditioned upon) the input text.
- Our encoder can be bidirectional, but our decoder must be unidirectional. Why?



# Encoder-Decoder with Transformers

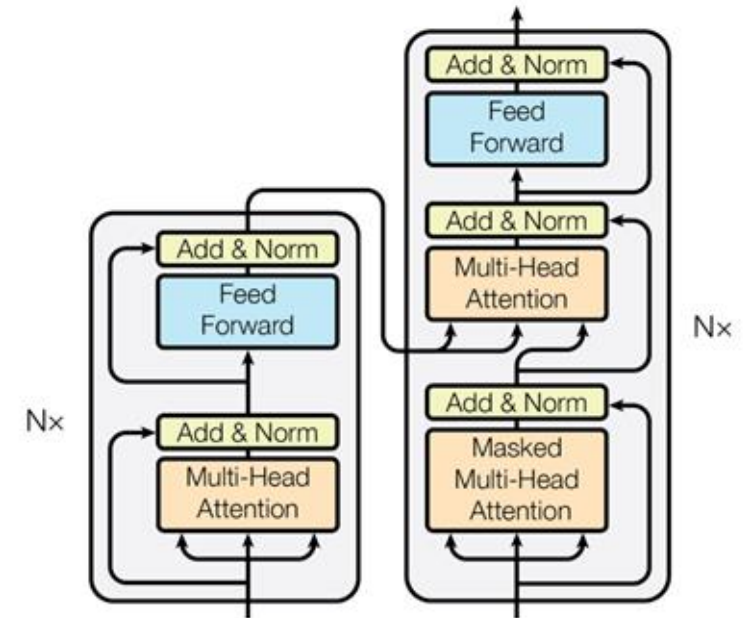
## Replace RNNs with Transformers

- Transformers are an architecture that wraps MultiHeadAttention(). We have some residual connections inside and some LayerNormalization() operations, with a couple of Dense layers on the output.
- We also need to create a special masking layer to prevent the Transformer from looking ahead in the target sequence.
- Our decoder uses cross-attention. It compares the representation of the Spanish output seen thus far to the entire English sequence representation.



# Translation with Self- and Cross-Attention

1. **First, Self-Attention in Spanish Sequence (words output so far), i.e., with masking.**
2. **Second, Self-Attention in English Sequence (entire sequence).**
3. **Finally, Cross-Attention Comparison.**
  - a. **Query:** The attended representation of the Spanish sequence (generated so far) acts as the query. "Given what I've generated in Spanish already, what information from the English sentence is most relevant to predicting the next word?"
  - b. **Key and Value:** The attended representation of the entire English sentence acts as both the "key" and "value." The "key" is used to compute attention weights (how much each English word should be attended to), and the "value" is the information that gets weighted and incorporated into the Spanish sequence.
4. **Apply the Attention Weights to English Representation:** The attention weights from comparing the query (Spanish) and key (English) are applied to the value (English). This results in a context vector representing the most relevant information from the English sentence for predicting the next Spanish word.



# Multi-Head Attention in Other Text Tasks

## BERT Embeddings

- **B**idirectional **E**ncoder **R**epresentations from **T**ransformers.
- Employs a similar (self-supervised learning) technique as Word2Vec. Mask 15% of words at random in any given sequence and then try to predict those using the words that are not masked.
- Word2vec is nice but yields static (generic) embeddings for a given word. BERT learns context-dependent representations. This can dramatically improve the predictive performance of, say, a text classifier.



# Common Use Case: Sentence Embeddings

## Mean Pooling of Contextual Word Embeddings

- For a given sentence, you can extract the contextual word embeddings from \*after\* the MultiHeadAttention layers and then average them.

## Pre-Trained BERT Classifier

- Or load a pre-trained BERT model's transformer and embedding layers (backbone / base layers) to pre-process your text and obtain contextual embeddings for a given prediction task.
- Or, load the embedding and attention layers, freeze them, and add your Dense layers on top like we did with pre-trained vision models.

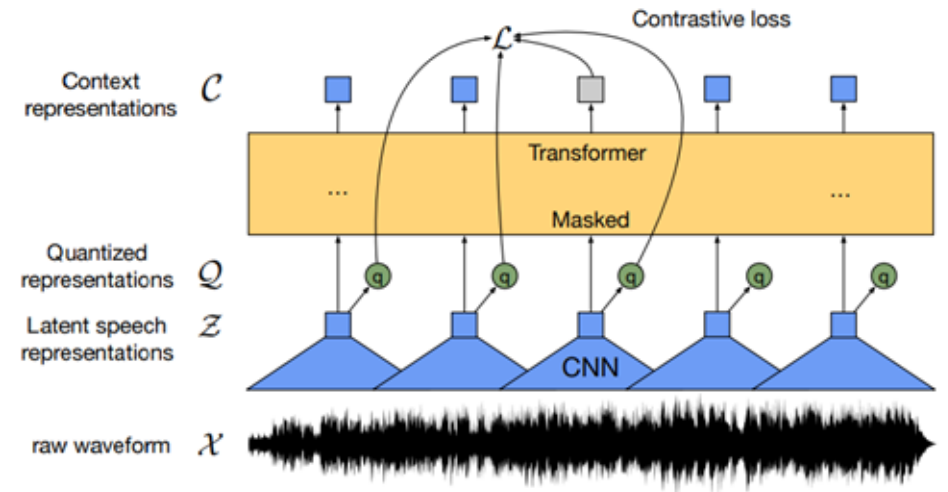
```
import keras_hub

tokenizer = keras_hub.models.Tokenizer.from_preset("roberta_base_en")
backbone = keras_hub.models.Backbone.from_preset("roberta_base_en")
```

# Multi-Head Attention for Audio Representations

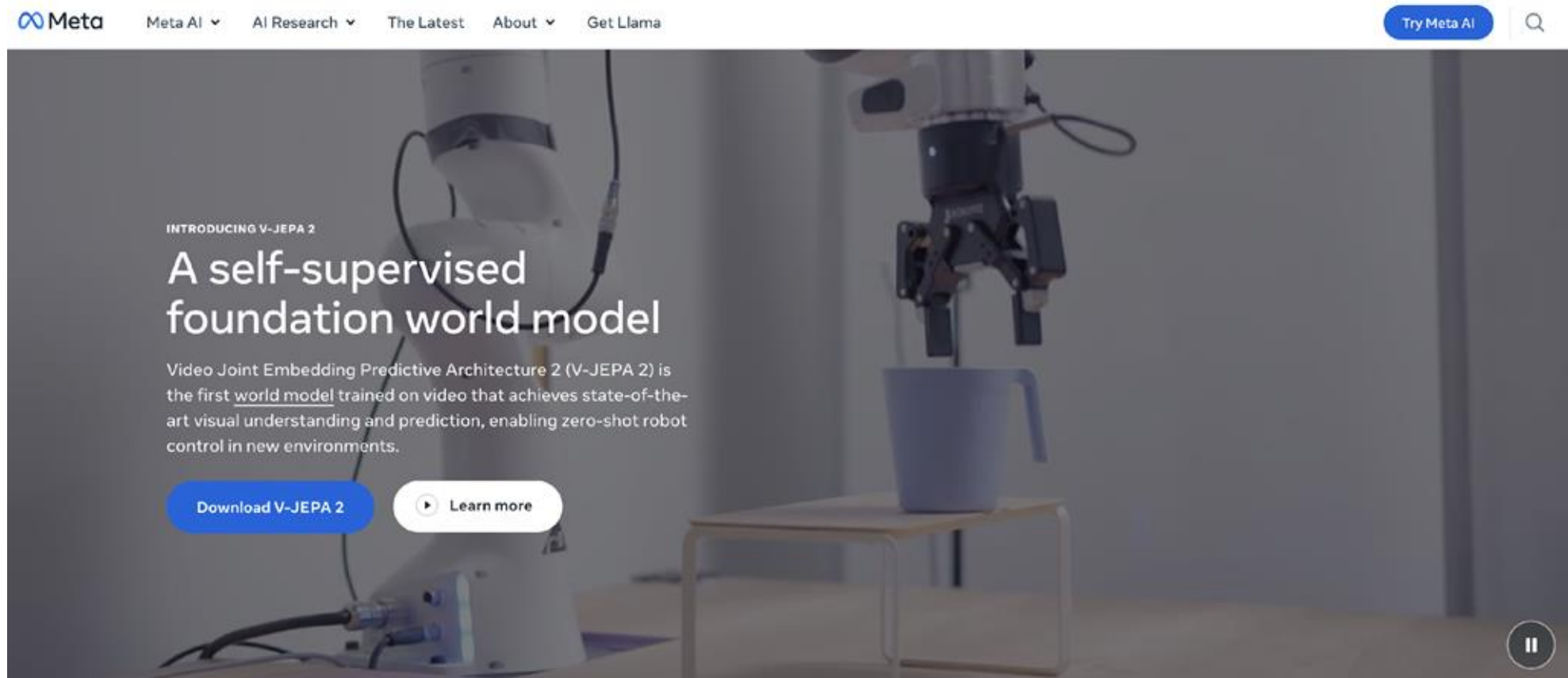
## Wav2Vec for Audio Understanding

- Wav2vec implements a self-supervised learning architecture with random masking of sequence input for audio.
- The model first obtains 1D convolutional feature maps per chunk/segment of raw audio signal (spoken words), yielding sequences of feature maps.
- A random subsample of feature maps is masked, and the model tries to predict them from the feature maps that remain visible. This model then yields audio embeddings highly representative of spoken semantic content in audio.





# 'Physics' Representations from Video



# CharRNN: Decoder-Only

## Training a Generative Language Model

- We train a model to predict the next token at every position in a sequence of observed input tokens.
- We will not use this exact model for inference; we will modify it slightly and add a feedback loop to make auto-regression possible.

```
embedding_dim = 256
hidden_dim = 1024

inputs = layers.Input(
    shape=(sequence_length,), dtype="int", name="token_ids"
)
x = layers.Embedding(vocabulary_size, embedding_dim)(inputs)
x = layers.GRU(hidden_dim, return_sequences=True)(x)
x = layers.Dropout(0.1)(x)
outputs = layers.Dense(vocabulary_size, activation="softmax")(x)
model = keras.Model(inputs, outputs)
```

# In Training: Loss at Each Sequence Position

## Training: Predict All Positions in Parallel

During training, we already know the full sequence. So, we compute loss at every position simultaneously. This is purely an efficiency trick — it does not mean the model "sees the future." Our model predicts one prediction per token in the sequence, in parallel.

### Example (character-level):

Position 0: model sees [S] → predicts 'h'

Position 1: model sees [S, h] → predicts 'a'

Position 2: model sees [S, h, a] → predicts 'k'

Position 3: model sees [S, h, a, k] → predicts 'e'

Position 4: model sees [S, h, a, k, e] → predicts 's'

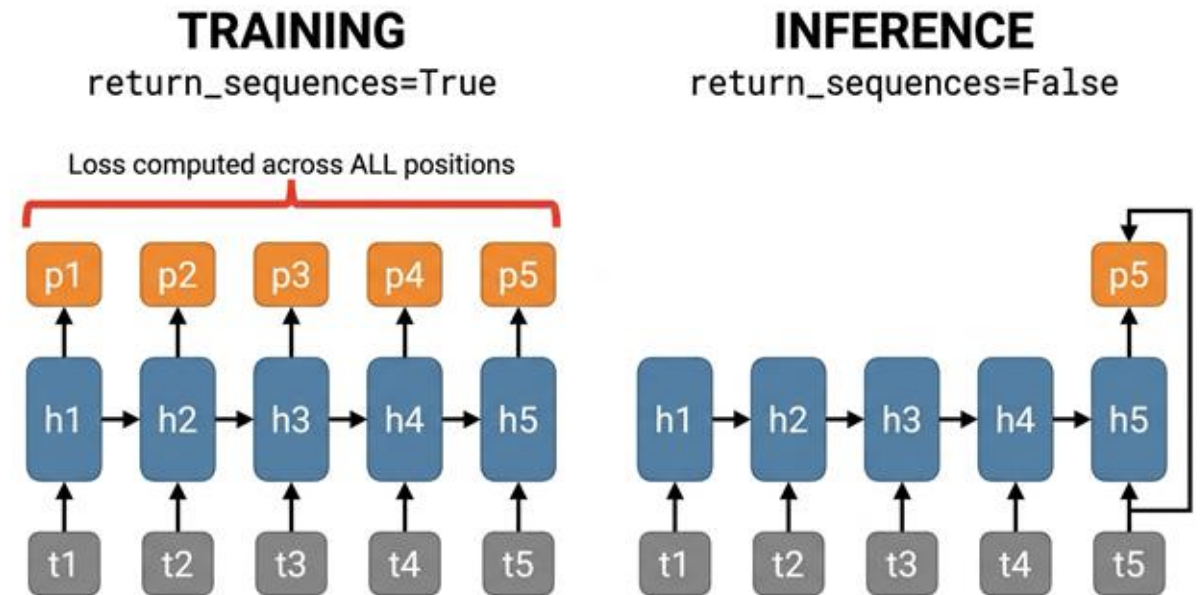
All 5 predictions happen in ONE forward pass, together.

Loss = average of all 5 cross-entropy values.

## Inference: One Token at a Time

At inference, we set `return_sequences = False`.

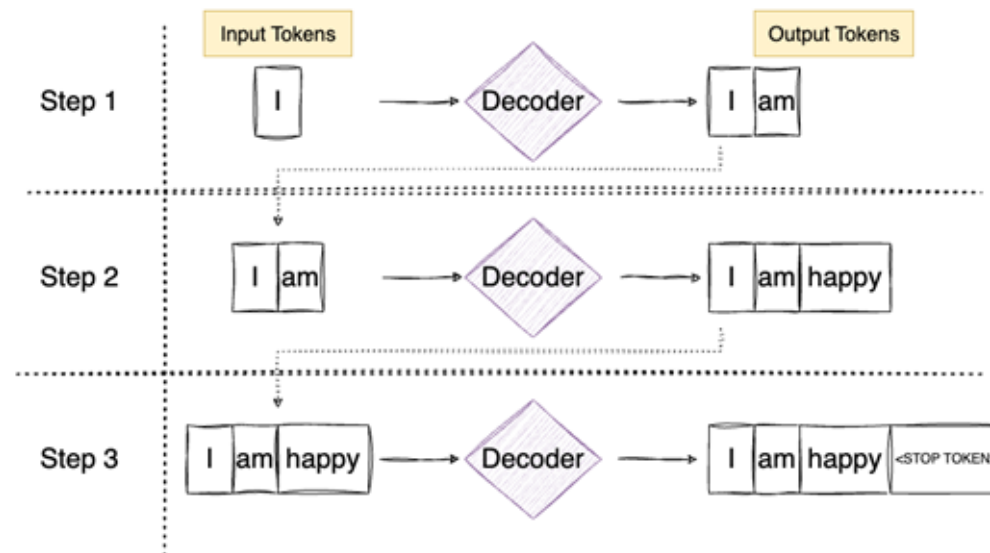
We generate one token only, feed it back as input, and repeat. Same weights, just a different loop.



# Generative Language Model

## Inference with Generative Language Models

- We modify the layer to only produce a single output token (`return_sequences = False`).
- We then wrap the new layers with a feedback loop that takes the last prediction and passes it as input to the model again along with the most recent embedding that captures ‘what we’ve done so far’.



# Where Did the Encoder Go?

Char-RNN is decoder-only: one RNN, no encoder, no attention. This works because input and output are trivially aligned — predict position  $t+1$  from position  $t$ . The hidden state  $h_t$  already encodes everything from characters 1 through  $t$ . No need to 'look back.'

Translation is *different*: input and output are different lengths, vocabularies, and orderings. You must read the whole input before writing any output.

The encoder-decoder split also solved a memory problem with RNN hidden states, which degrade over long sequences. The encoder saves all the input hidden states; attention lets the decoder look back (like Bahdanau attention).

Ultimately, the encoder-decoder architecture compensates for RNN limitations, not task requirements. When the task is aligned (shift by 1), a single RNN is both simpler and better. With transformers, even in the case of translation tasks, we no longer need the encoder-decoder architecture (given enough pre-training).

# Generative Pre-trained Transformers

## Generative Pre-trained Transformers (GPTs)

In 2018, not long after “Attention Is All You Need,” OpenAI put out a paper about the efficacy of scaling data and compute with a mixture of basic ingredients: i) vast amounts of highly varied text, ii) a generative language architecture, and iii) transformers (oh, and RLHF).

- GPT-1 (2018) had 117 million parameters trained on 10 million tokens.
- GPT-2 (2019) had 1.5 billion parameters and was trained on 10 billion tokens.
- GPT-3 (2020) had 175 billion parameters and was trained on 300 billion tokens.

---

### Improving Language Understanding by Generative Pre-Training

---

Alec Radford  
OpenAI  
alec@openai.com

Karthik Narasimhan  
OpenAI  
karthikn@openai.com

Tim Salimans  
OpenAI  
tim@openai.com

Ilya Sutskever  
OpenAI  
ilyasu@openai.com

#### Abstract

Natural language understanding comprises a wide range of diverse tasks such as textual entailment, question answering, semantic similarity assessment, and document classification. Although large unlabeled text corpora are abundant, labeled data for learning these specific tasks is scarce, making it challenging for discriminatively trained models to perform adequately. We demonstrate that large gains on these tasks can be realized by *generative pre-training* of a language model on a diverse corpus of unlabeled text, followed by *discriminative fine-tuning* on each specific task. In contrast to previous approaches, we make use of task-aware input transformations during fine-tuning to achieve effective transfer while requiring minimal changes to the model architecture. We demonstrate the effectiveness of our approach on a wide range of benchmarks for natural language understanding. Our general task-agnostic model outperforms discriminatively trained models that use architectures specifically crafted for each task, significantly improving upon the state of the art in 9 out of the 12 tasks studied. For instance, we achieve absolute improvements of 8.9% on commonsense reasoning (Stories Cloze Test), 5.7% on question answering (RACE), and 1.5% on textual entailment (MultiNLI).

# GPTs (and LLMs Generally) Have Many Problems

## Various Challenges with LLMs (Among Many Others)

- Hallucination: They produce plausible text as output, not necessarily truthful or correct text. Therefore, they are highly dependent on the quality of training data.
- Prompt Brittleness: Rewording a prompt even slightly can lead to considerable changes in model response. For example, if you ask the same question of an LLM in different languages, you can get very different answers (conceptually and semantically)!
- Memorization: Assessing the performance of LLMs on various task benchmarks is quite challenging because it is often tough to assess whether the model has seen the problem previously and is merely looking up the answer in training data. This affects whether performance reflects how a model might perform on an entirely new task.

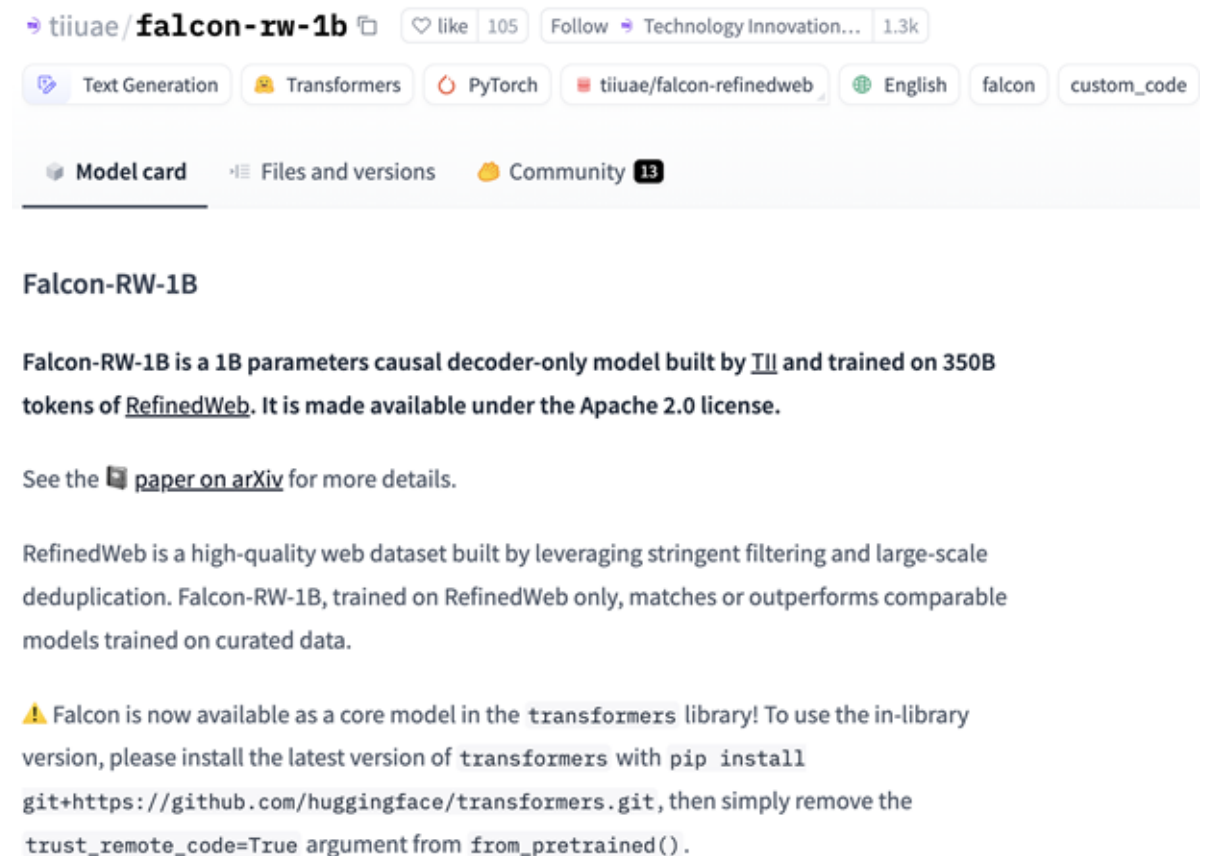


# Lots of Pre-Trained LLMs Around

## Loading Pre-trained Models from Keras Hub

Check out what is available on the Keras hub. Many open-source and free-to-use models are there, though many are too large for a Colab notebook.

Kaggle hosts some of the models. For a given model, you may need to visit Kaggle and create an API key, complete a request form, or agree to terms and conditions.





# Questions?